

What Bounded Retries Can Guarantee: Calls, Protected Work, and Partial Repair

ANONYMOUS AUTHOR(S)

A retry limit bounds failed calls, but fallback may move all remaining computation under protection. For specified completion operations on n persistent whole-kernel jobs, we derive exact frontiers under $n + r$ calls. Cached completion attains the best protected-work curve for every hidden external-write bound B ; without it, a universally capped policy can be forced to protect all work at $B = r$. For partial repair with no call charge and positive retry allowance, one policy is optimal for all budgets exactly when each job can suffer maximum damage from one write. Call charges and precedence change this boundary. A finite construction finds a policy with the least worst-case cost excess over all informed budget-specific optima, including when exact common optimality fails. Java realizes the whole-kernel interface under stated assumptions. Roslyn modifications preserve checked events, snapshots and compiler projections, while stronger rebase operations retain unmodeled merge costs. Authored arrivals show mixed timing effects. These results identify operation-specific resource guarantees and information requirements.

CCS Concepts: • **Software and its engineering** → **Concurrent programming structures**.

Additional Key Words and Phrases: concurrent programming, retries, resource guarantees, partial repair, online policies

1 Introduction

In optimistic update loops, a caller prepares outside protection and validates before publication. Switching to protected execution after finitely many failures bounds failed calls, but can protect all remaining computation. Which bounds on calls and protected work can be met together, and which completion operations are necessary?

Bounded speculation followed by locking is established practice. glibc’s enabled x86 elision path permits at most three transaction starts before falling back to normal locking; HotSpot exposes RTMRetryCount [22, 23]. HTM can abort without a write and discards speculative work, so it motivates finite attempt budgets but does not instantiate our retained-preparation model. We study validate-and-complete interfaces with per-update, shared-batch or combined retry allowances. Local allowances sum to a batch cap; a shared allowance permits redistribution (Proposition 3). These are authored design requirements, with no assumed latency SLA.

Consider dependent transformations with works 8 and 16 and a two-call cap. Each call must complete a job. Fresh completion protects all 24 units. Cached completion prepares outside, then reuses or recomputes within one completing callback. Its worst protected work is 0, 16 or 24 at write bounds 0, 1 or 2, without receiving that bound (Figure 1). At one write, however, worst total work rises to 40 from fresh completion’s 24.

Write bound B	Worst protected work L		Worst total work W	
	Two modes	Three modes	Two modes	Three modes
0	24	0	24	24
1	24	16	24	40
2	24	24	24	48

Fig. 1. Two dependent jobs with works 8 and 16 and $r = 0$. A cached callback completes on either comparison outcome; a cheap mismatch requires a new preparation. Both fixed policies satisfy $Q \leq 2$ in every finite-write environment; neither receives B . Values follow from the model, not timing measurements.

For works $(1, 2, 3, 5)$, a six-call cap allows $r = 2$ failures. Cached completion protects at most eight units at four writes; without it, a universally capped policy may protect all eleven at two writes. Necessity is relative to the specified operations. Protected work contributes to shared-exclusion blocking, but excludes waiting and predicts no latency. Section 7 isolates this contribution with a shared-bin probe and reports mixed arrival timings.

Our results concern call/protection frontiers, repair information boundaries and minimum uniform loss, assessed in Java/Roslyn. Fallback, contention control, synchronization choices and robust DP predate them [5, 8, 11, 12, 20, 21].

The curve needs no estimated write bound; a fixed protection ceiling does. Source checks do not establish adoption.

2 Completion Interfaces and Resource Contracts

Write $\mathbb{N} = \{0, 1, \dots\}$. Jobs $J = \{1, \dots, n\}$, $n \geq 1$, form a finite DAG with predecessor set $\text{Pred}(i)$. Readiness requires all predecessors completed. Completion records an exact immutable output and persists despite later external writes. The objective is one successful foreground transformation per job, not a simultaneous snapshot or transaction. Policies are deterministic and causal; no randomized expected-cost guarantee is claimed.

Each ready job has a pure, opaque, whole kernel receiving its own-input identity and exact completed predecessor outputs. Every execution performs its fixed work $w_i > 0$, whether inside or outside protection. Partial/incremental evaluation, preparation before readiness and discharge by external work are excluded in this interface. A preparation is tied to its own-input identity and predecessor tuple. Captured outputs cannot be reconstructed by later reads of the mutable map.

States, histories and turns. Write a state as $(x, \mathcal{M}, \mathcal{R}, \ell)$: live own-input identities x , the partial map $\mathcal{M}$ of completed jobs to immutable outputs, a finite snapshot/record store $\mathcal{R}$, and program-local state ℓ . A record contains its job, captured identity, predecessor tuple and computed result. Records may be retained, copied and selected, but not fabricated or retagged without the corresponding computation. In Table 1, U means no selected record, C a selected record, possibly stale, and D permanent completion. Failed cheap calls clear the selection, leaving retained copies available. After every finite history, each unfinished job admits a legal own-key write with an identity never used for that job. Writes preserve completed outputs.

A finite visible history h lists program actions, returned values and completion events. The program chooses a ready job and operation, or an unprotected inspection, preparation or local record operation. After that choice, the environment chooses a finite word of legal writes, then the operation executes. This environment move is the *gate*; it precedes comparison even within a callback. The mode/job is fixed before the gate; any later record selection uses only the prior store. Comparison and completing computation/publication are indivisible to writes; protection ends on return. Inspection honestly returns the live entry. Preparation computes a whole kernel from a captured immutable snapshot and completed parents; old captured snapshots are also allowed. Fixing the capture point, its pure body has no live read or visible interaction. Writes after capture can be replayed in order at the next gate, preserving the returned record and every subsequent observation. Only that pure body is contracted; capture is not moved.

Let W count all foreground kernel work, L its protected part, and Q validation, publication and completion calls, including failures. Preparations, initialization and external-writer calls are excluded from Q . These are separate resources. Common call overhead is excluded from L . Proposition 4 charges it explicitly in this interface; Section 4.2 treats the distinct repair interface. Let $\Omega_k(T)$ sum the largest $\min(k, |T|)$ works in T . Write $\Omega(T)$ for the total; omitted sets mean J . Set $\Omega_0 = 0$. These aggregate/allocation objects are inherited [9, 10].

Table 1. Whole-kernel operations. Every validation or callback adds one to Q ; preparation adds none. Entries show successor state and added (W, L) . Each callback holds protection only within that operation.

Operation	Start	Match	Mismatch
Prepare	U	C, $(w_i, 0)$	—
Cheap publication	C	D, $(0, 0)$	U, $(0, 0)$
Fresh callback	U	D, (w_i, w_i)	—
Cached callback	C	D, $(0, 0)$	D, (w_i, w_i)

A *cheap* call publishes on a match and returns without completion on a mismatch. Conditional publication and validation-only callbacks have this same resource transition; their Java implementations appear in Section 5. The label concerns kernel work, not elapsed time. Two modes provide cheap publication and fresh completion; three also provide cached completion. Failed validation leaves the live entry unchanged. Inspection and local record operations cost no Q, W, L and never publish or retain a guard. Direct acquire/release, protection surviving return, cross-job callbacks and outside completion are absent from this endpoint contract. These restrictions define the comparison; they do not assert completeness for arbitrary Java code or lock APIs. Each selected operation terminates. A play that never completes all jobs, including infinite free actions, has infinite objective; fairness of selected operations is an explicit assumption.

Strategies and admissibility. An informed strategy is a deterministic function $P(B, h)$; an unaware strategy is $P(h)$. Let $\mathcal{E}_b$ contain gate strategies using at most b writes along every play. For mode family j , $\mathcal{P}_j(B, r)$ consists of strategies completing with $Q \leq n + r$ against every $E \in \mathcal{E}_B$. By contrast, $\mathcal{U}_j(r)$ consists of one B -independent strategy satisfying that cap for every $b \geq 0$ and $E \in \mathcal{E}_b$. Every finite-write play against a fixed program is preserved by capping its environment at that play's write count. Worst cost at budget B is $\sup_{E \in \mathcal{E}_B} L(P, E)$; the two minima use different admissible classes.

LEMMA 1 (FRESH-WRITE AND FINITE-ADVERSARY FACTS). *In Section 2's game, a fresh own-key write at a selected i -operation's gate invalidates every retained i -record. Any concrete prefix has both a no-write and a fresh-write continuation whenever that extra write respects the environment's budget. An adversary bounded by b writes on every play forces any $P \in \mathcal{P}_j(b, r)$ to finish every job, regardless of free inspections or record retention.*

PROOF. Every retained i -record carries an identity previously captured from i ; the fresh write uses an identity outside that set. Copying or selecting records cannot change this. All choices before that gate, including inspection results and the selected job/mode, coincide in the two continuations. Only the next environment move differs, so no prior observation removes the fresh-write continuation. This does not erase the observation history or assert an observation-free normal form.

A globally b -bounded adversary belongs to $\mathcal{E}_b$. A play on which the program indefinitely inspects, prepares, or works on other jobs while leaving i unfinished violates admissibility on that very finite-write play. Consequently each job eventually completes. Its completing operation is either a fresh callback or a comparison using a prior record, exhaustively by Table 1. In the first case its whole kernel is protected without a write; in the second the fresh-write argument applies. No truncation of an unbounded adversary is used. $\square$

3 Universal Call and Protected-Work Guarantees

For unfinished jobs S , let $\mathcal{A}(S) = \{i \in S : \text{Pred}(i) \cap S = \emptyset\}$ denote the ready jobs.

Table 2. Main resource and information symbols. Later sections define their derived values.

n, B, r	Job count, external-write bound, shared allowance of noncompleting calls
Q, W, L	Counted calls, total kernel work, protected kernel work
Ω_k, Ω	Sum of the k largest job works; sum of all job works
r_i, s, R	Local allowance, simultaneous shared allowance, sum $\sum_i r_i$
$D, \Gamma, G(b)$	Observed dirty set, write footprints, maximum damage from at most b writes
W^*, W_i, σ_i	Maximum repair damage (one job / job i); writes needed to saturate job i
q, κ, μ	One-job call cap, charge per call, common completion charge

THEOREM 1 (PROTECTED WORK UNDER A CALL CAP). *In Section 2's interface, fix integers $B, r \geq 0$. Among programs supplied with B that complete all jobs with $Q \leq n + r$ in every environment with at most B writes, the minimum worst protected work is*

$$L_3^*(B, r) = \Omega_{\min((B-r)_+, n)}, \quad L_2^*(B, r) = \begin{cases} 0, & B \leq r, \\ \Omega, & B > r, \end{cases}$$

with and without cached completion, respectively. Any DAG is permitted.

PROOF. At most r calls may fail to complete. Freshly prepared cheap attempts have disjoint read-to-comparison intervals; each failure needs a distinct write. If $B \leq r$ they give $Q \leq n + B$, $L = 0$. Otherwise switch after r failures to cached completion: at most $B - r$ writes remain, giving the stated bound. All-protected gives the two-mode upper bound.

For the two-mode lower bound when $B > r$, reject each cheap call with a fresh own-key write, stopping writes after the $(r + 1)$ st rejection. This uses at most $r + 1 \leq B$ writes. An admissible program cannot incur that many noncompleting calls, so all its jobs complete under protection.

For three modes, put $k = \min(B - r, n) > 0$ and fix the k heaviest jobs as targets. The adversary maintains f , the number of noncompleting target comparisons, and p , the number of completed targets. Initially both are zero. It writes only at a target's comparison gate, with a fresh identity, while $f \leq r$ and $p < k$. It stops all writes permanently when $f = r + 1$ or $p = k$. Counters follow actual outcomes: f increases on a noncompleting target comparison and p on every target completion. While writing is active, cheap target calls fail and cached ones recompute. Fresh completion protects w_i without a write. After stopping, calls follow their ordinary semantics.

Let v be the writes so far. At each program node, after the preceding operation returns, induction gives $v \leq f + p$: each write led to a failure or a cached completion; fresh completion only increases p . Immediately before any further write, $f \leq r$ and $p \leq k - 1$, giving $v + 1 \leq f + p + 1 \leq r + k \leq B$. Once stopped, no write is added. Thus this is unconditionally an $\mathcal{E}_B$ strategy, including plays that violate the call cap.

For an admissible program, $f = r + 1$ is impossible: completion requires n completing calls in addition to those failures, already exceeding $n + r$. Thus $f \leq r$ throughout its play. Lemma 1 forces every target eventually to complete; each such completion is fresh or is a cached mismatch, both protecting its whole work. Consequently $p = k$ and $L \geq \Omega_k$. Free inspections, stale selections and retained copies cannot evade either case. Predecessors still require their own completing calls, already included in the mandatory n . A nontarget predecessor changes neither f nor p and receives no write from this adversary. A target predecessor is counted exactly once in p . Thus dependencies delay targets without changing the write or call accounting. $\square$

THEOREM 2 (ONE POLICY FOR EVERY WRITER BUDGET). *Let $\mathcal{U}_j(r)$ contain the j -mode programs that receive no B and complete with $Q \leq n + r$ in every finite-write environment. Set $U_j(B, r) =$*

$\inf_{P \in \mathcal{U}_j(r)} \sup_{\# \text{writes} \leq B} L(P, E)$. Then

$$U_3(B, r) = L_3^*(B, r), \quad U_2(B, r) = \Omega \mathbf{1}[B \geq r].$$

One policy per mode family attains these values simultaneously for all B and positive works on any DAG, using only readiness and a global cheap-failure counter.

PROOF. Switch after r cheap failures to a fresh read/preparation with cached completion (three) or to fresh protected completion (two). This uses at most r extra calls universally. At $B < r$ no protection is needed; at $B = r$ cached completions after switching still match. Theorem 1 supplies the remaining upper and three-mode lower bounds.

For two modes when $B \geq r$, reject every cheap call with a fresh own-key write until r rejections, then stop. Only protected calls complete before this. Afterwards, another rejectable call could fail in a different finite environment, violating the call cap. Thus all jobs complete under protection with at most $r \leq B$ writes. That additional hypothetical write is absent from the lower-bound execution. Retained preparations and inspections cannot prevent a fresh write before comparison. $\square$

For equal works and $B > r$, the ratio is $n/\min(B - r, n)$. At $B = r$, only an informed two-mode program knows no write remains and may risk another cheap call. The unaware class must also be safe in extensions with another write, causing the boundary shift. A deployment that supplies a certified bound and requires the cap only within it instead uses $\mathcal{P}_j(B, r)$; that is a different information contract.

Readiness constrains when a target is attempted, but persistence requires every target eventually. Thus DAG edges do not change this frontier. They become consequential with repair operations and call costs (Section 4.3).

3.1 Local Caps and a Shared Cap Can Be Required Together

Suppose job i may incur at most r_i noncompleting calls, and the batch at most $s \leq R = \sum_i r_i$. These caps hold in every finite-write environment. Define the weighted subset value

$$A_c(B) = \max_{T \subseteq J: \sum_{i \in T} c_i \leq B} \sum_{i \in T} w_i,$$

allowing zero-cost items. This is a knapsack value; the contribution is identifying the write cost that activates each job's protection.

PROPOSITION 3 (LOCAL AND SHARED RESOURCE CONTRACTS). *The minimum worst protected work under both caps is*

$$U_2(B; \mathbf{r}, s) = \max\{A_r(B), \Omega \mathbf{1}[B \geq s]\},$$

$$U_3(B; \mathbf{r}, s) = \max\{A_{r+1}(B), \Omega_{(B-s)_+}\}.$$

One policy attains each curve simultaneously for every B : switch to guaranteed completion when either the current local allowance or the shared allowance is exhausted. Its worst call count is $n + \min(B, s)$.

PROOF. For a local lower bound, target a maximizing subset. Reject each target's cheap calls up to its local cap. In two modes another rejectable call is then universally inadmissible, so the target completes protectively using at most r_i actual writes. In three modes, one additional write invalidates its eventual cached comparison; an earlier fresh completion already pays w_i . Thus each target costs at most $r_i + 1$ writes. The shared lower bounds follow from Theorem 2, since adding local constraints cannot enlarge its policy class.

For the switching policy, if fewer than s failures occur, a protected job has exhausted its own cap: it requires r_i writes in two modes, and $r_i + 1$ in three. If s failures occur, two modes pay at most Ω ; in three, every protected completion still needs its own additional mismatch write, so at

most $(B - s)_+$ jobs are protected. These two cases give the maximum above. A fresh preparation after every rejection makes the intervals disjoint. Distributing $\min(B, s)$ cheap failures among the local allowances attains the call bound; its maximum need not coincide with the protection maximum. $\square$

With only local caps, $s = R$ gives $U_2 = A_r(B)$ and $U_3 = A_{r+1}(B)$. Uniform $r_i = r \geq 1$ yields $\Omega_{\lfloor B/r \rfloor}$ and $\Omega_{\lfloor B/(r+1) \rfloor}$, with $Q \leq n + nr$; at $r = 0$, the values are Ω and Ω_B , with $Q = n$. Reusing a rejected preparation loses the extra-write requirement. Sharing allowance R permits weakly less worst protection than reserving it locally.

3.2 Choosing Speculation When Every Call Has a Cost

Let each counted call cost $\kappa \geq 0$ and minimize $C = L + \kappa Q$. Let $C^*(B, r)$ be the minimum worst cost over the full B -informed program class of Theorem 1, with only the shared cap. Mode choice remains *before* comparison; the partial-repair interface below allows a different observation/action order. Write $C(P, B)$ for policy P 's worst cost at budget B .

PROPOSITION 4 (LOSS OF THE GLOBAL-THRESHOLD POLICY). *The three-mode policy P_r that freshly prepares before each attempt and switches after r cheap failures to cached completion has*

$$C(P_r, B) = n\kappa + \kappa \min(B, r) + \Omega_{(B-r)_+},$$

$$0 \leq C(P_r, B) - C^*(B, r) \leq \kappa \min(B, r) \leq r\kappa.$$

The $r\kappa$ bound is sharp for this policy.

PROOF. The call and protection upper bounds follow from Theorem 2. They are attained together: reject the first $\min(B, r)$ cheap calls, then use remaining writes to invalidate cached completions of the heaviest jobs. Every competitor has a resource lower-bound execution with both $L \geq \Omega_{(B-r)_+}$ and $Q \geq n$. Thus $C^* \geq n\kappa + \Omega_{(B-r)_+}$ on that execution. For one job and $B = r + 1$, invalidating every comparison forces protected completion. Immediate cached completion is optimal at $\kappa + w$, while P_r pays $(r + 1)\kappa + w$. $\square$

The bound compares a specified policy with an informed comparator. A caller may instead want the best possible loss without ever learning B . For one job this question has an exact answer.

PROPOSITION 5 (EXACT ONE-JOB MINIMAX ADDITIVE LOSS). *For one job with work $w > 0$, let P range over all deterministic programs that receive no B and complete with $Q \leq r + 1$ in every finite-write environment. Then*

$$\inf_P \sup_{B \geq 0} (C(P, B) - C^*(B, r)) = \min\{(w - \kappa)_+, r\kappa\}.$$

Always-cached completion attains the first term for $r \geq 1$; P_r attains the second. Choose the smaller, without estimating B .

PROOF. The informed one-job value is $\kappa + \min(B\kappa, w)$ for $B \leq r$, and $\kappa + w$ for $B > r$. For the first case, cheap attempts and immediate cached completion give the two upper bounds. For a lower bound, count all noncompleting calls in a . While $a < B$, invalidate each selected usable comparison with at most one fresh write. Actual writes are at most a ; a cached completion can therefore be invalidated within $a + 1 \leq B$ writes. Fresh completion already pays w . If $a = B$ is reached, stop writing: any subsequent completion has at least $B + 1$ calls, regardless of its mode. For $B > r$, invalidating through the cap uses at most $r + 1 \leq B$ writes and forces protected completion.

Now consider an unaware program with $r \geq 1$ and $0 < \kappa < w$. Again invalidate each chosen usable comparison, and let $a \leq r$ count all noncompleting calls before its first fresh/cached completion. If fresh, witness budget $B = a$ gives regret at least $a\kappa + w - \min(a\kappa, w) \geq w$. If cached and

$a < r$, one more write and $B = a + 1$ give regret at least $a\kappa + w - \min((a + 1)\kappa, w) \geq w - \kappa$. If cached and $a = r$, use $B = r + 1$ to get $r\kappa$. These are upper bounds on actual writes, not assumed equalities. Stop the adversary after at most $r + 1$ noncompleting calls; the cap and finite-write completion exclude an infinite avoidance run. Fresh identities invalidate retained preparations after mode selection, so free inspections do not change the argument.

Always-cached has worst regret $(w - \kappa)_+$ for $r \geq 1$. Substituting the informed curve above into Proposition 4 gives worst regret $r\kappa$ for P_r . Nonnegativity covers $r = 0$, $\kappa = 0$ and $\kappa \geq w$. $\square$

This specializes classical cost balancing [34] and capped rent-or-buy [26]. For a serial threshold $t \leq r$, subtracting the inevitable completion toll gives cost $B\kappa$ for $B \leq t$ and $t\kappa + w$ otherwise. Beyond $B = r$, the call cap also forces the informed comparator to complete protectively. Threshold selection and regret are inherited ideas; the proof additionally covers inspections, retained records and fresh completion. When $0 < \kappa < w$, immediate cached completion minimizes loss iff $w \leq (r + 1)\kappa$; otherwise use all retries. With $w = 8, r = 2$, tolls 1 and 3 select retries and immediate completion, respectively. The units are supplied costs; general whole-kernel workflow minimax loss remains open. The repair interface below has a separate finite solution.

4 When Partial Repair Makes the Budget Valuable

Whole-kernel recomputation makes one invalidation as damaging as many. We next study an interface that reuses unaffected components, and characterize when partial damage destroys a common optimum. Roslyn merges are not assumed to satisfy this cost law.

Repair interface and observations. One job consists of m components with positive weights w_c . A write changes fresh versions on one nonempty footprint from a fixed finite family Γ . Footprints may overlap and repeat; each occurrence costs one write. Outside protection, preparation reads and computes every component anew. A protected call observes the dirty set D , the union of footprints since that preparation. It either accepts, repairs exactly D and publishes the job, or rejects and prepares again outside. An empty observation can complete with no repair. No partial publication, free dirty-set query, retained protection or cheaper completion operation is available. A fresh whole-job completion costs at least repairing any dirty set. Calls and preparations terminate.

The turn order is preparation, an environment word of footprints, observation D , then the accept/reject choice within that same counted call. No write intervenes between observation and decision. A visible history lists job choices, observations and decisions, including current D ; repeated writes and word lengths are hidden. Informed strategies additionally receive B . Completion is required within $q \geq 1$ calls in every finite-write execution. One preparation per call permits at most q preparations. The objective is protected repair work, excluding outside preparation and common call overhead. Let

$$G(b) = \max_{0 \leq t \leq b, Y_j \in \Gamma} \sum_{c \in Y_1 \cup \dots \cup Y_t} w_c, \quad W^* = w\left(\bigcup \Gamma\right).$$

Then $G(0) = 0$, G is nondecreasing and subadditive, and $G(m) = W^* > 0$. Unreachable components do not contribute to W^* . Singleton footprints give the prefix sums of weights sorted in decreasing order. A general footprint family need not have a cheap profile: computing these unions is part of the input-processing cost.

THEOREM 6 (EXACT VALUE AND INFORMATION BOUNDARY FOR REPAIR). *For a controller supplied with B , minimum worst repair work is*

$$K_q(B) = G(\lfloor B/q \rfloor).$$

Among controllers receiving no B that attain $K_q(B) = 0$ for every $B < q$, the least possible worst-work envelope is

$$T_q(B) = G((B - q + 1)_+).$$

One policy attains this envelope for all B . For $q \geq 2$, a single controller is optimal for every B if and only if $G(1) = W^*$. For $q = 1$, such a controller always exists.

PROOF. Put $k = \lfloor B/q \rfloor$. The aware policy accepts work at most $G(k)$ and otherwise rejects while a call remains. An observation with greater work requires at least $k + 1$ writes. Rejecting $q - 1$ such observations exhausts all opportunities for a q th: $q(k + 1) > B$. Thus forced acceptance on the last call still meets $G(k)$. This policy also satisfies the call cap outside its supplied budget, where the work guarantee is not asserted. Conversely, a word of at most k footprints realizes $G(k)$. Repeat it after every preparation. Any accepting call pays $G(k)$, and q repetitions use at most $qk \leq B$ writes. Fresh whole-job completion is no cheaper.

For the unaware lower bound, write once in each of the first $q - 1$ preparation intervals. After observation $j < q$, the history is compatible with only j writes in total. Accepting positive damage, or fresh completion, would violate that budget's zero optimum. Hence the policy must reject all $q - 1$ times. In its final interval spend up to $B - q + 1$ writes realizing $G(B - q + 1)$. No retry remains. The matching policy rejects its first $q - 1$ nonempty observations, then accepts; earlier empty observations complete for zero work.

Every simultaneous optimum meets the preceding zero-work requirements. If $G(1) < W^*$, let $k \geq 2$ be the first index with $G(k) > G(1)$, and set $B = q - 1 + k$. Then $1 \leq \lfloor B/q \rfloor < k$, so $K_q(B) = G(1) < G(k) = T_q(B)$. Conversely, if $G(1) = W^*$, both expressions are zero for $B < q$ and W^* otherwise. For $q = 1$ their arguments coincide. $\square$

The boundary concerns write damage: a footprint covering all reachable work preserves simultaneous optimality. With separate-component writes, further damage can make budget knowledge change the optimal acceptance threshold.

For two unit components, singleton writes and $q = 2$, $K_2(B)$ at $B = 0, 1, 2, 3$ is $0, 0, 1, 1$. Optimality at $B = 1$ forces an unaware policy to reject its first dirty component. Two subsequent writes make both components dirty; its last call costs 2 at $B = 3$. The aware policy at that budget accepts one dirty component immediately and retries only two-component damage. Figure 2 isolates the shared observation behind this separation.

Budget	First observation	Unaware decision	Possible final observation/cost
$B = 1$	Only component A dirty	Must reject to achieve optimum 0	Empty; cost 0
$B = 3$	Only component A dirty	Same observed prefix: reject	Both A and B dirty; cost 2

Fig. 2. Two unit components and two calls. Before the first decision the controller cannot distinguish these budgets. At $B = 3$, an informed controller can accept the first unit of damage and reject only full damage, guaranteeing cost 1. The unaware controller's forced first rejection exposes it to cost 2. No timing assumption is used.

Quantifying the loss without a budget. For repair work alone, a finite competitive ratio requires zero work when $K_q(B) = 0$. Theorem 6 therefore identifies the exact optimal ratio as $\sup_{B \geq q} T_q(B)/K_q(B)$. Subadditivity gives an upper bound q : with $k = \lfloor B/q \rfloor$, $B - q + 1 \leq qk$, so $G(B - q + 1) \leq qG(k)$. It is sharp for sufficiently many unit singleton components at $B = 2q - 1$,

where the ratio is q . Thus the loss can grow with the call allowance; it is not a constant-price perturbation of whole-kernel optimality.

4.1 A Shared Retry Allowance Across a Workflow

The same boundary extends beyond a single job. Consider n persistent jobs with a DAG, each with its own component/footprint family. A ready job is freshly prepared before each attempted observation; no other preparation is retained. A write affects the currently attempted job, with no cross-job footprint. After a rejection the next ready job may be selected anew. The total call cap is $n + r$, with a shared r ; acceptance pays that job's dirty-component work and permanently completes it. Write $W_i = G_i(\infty)$, $\sigma_i = \min\{b : G_i(b) = W_i\}$, $\Sigma = \sum_i \sigma_i$, and $W^* = \sum_i W_i$. Let $K(B)$ be the known-budget minimum worst repair work for this interface.

THEOREM 7 (WORKFLOW INFORMATION BOUNDARY). *For $r \geq 1$, one policy simultaneously attains $K(B)$ for every B iff $G_i(1) = W_i$ for every job. This holds both for a fixed order and for choices among DAG-ready jobs. Under the condition,*

$$K(B) = \text{sum of the largest } \min((B - r)_+, n) \text{ values } W_i.$$

For $r = 0$, accepting every observation is simultaneously optimal for arbitrary footprints. More generally, the first budget at which $K(B) = W^$ is*

$$B_{\text{sat}} = \Sigma + r \max_i \sigma_i.$$

PROOF. If every job has a one-write maximum, reject the first r nonempty observations globally and otherwise accept. Early acceptances are clean; after r rejections, at most $(B - r)_+$ jobs can accept positive damage. To force the matching lower bound, target that many heaviest jobs, writing once before each target observation to cause maximum damage. A target completion costs W_i , and at most r rejections add writes, so $r + k \leq B$ writes suffice for k targets. An attempted $(r + 1)$ st rejection would itself violate the call cap within budget; truncate the adversary there. Selection and precedence do not avoid completing the targets.

For necessity, all $K(B)$ with $B \leq r$ are zero. A simultaneous optimum must therefore reject when one write is issued before each of its first r observations: after observation t , the history is compatible with only $t \leq r$ writes in total. No job has yet completed. Now spend σ_i writes to saturate each job on its next attempt. No rejection remains, so the policy pays W^* with $B_0 = \Sigma + r$ writes. If some $\sigma_j \geq 2$, a competing policy reserves all r rejections for fully dirty observations of job j and accepts everything else. Full total work would require $\Sigma + r\sigma_j > B_0$ writes, as every invocation uses a fresh preparation. Its worst cost is therefore strictly below W^* ; the attainable cost set is finite. This contradiction proves necessity even in a prescribed order.

At $r = 0$, every call must complete and acceptance dominates fresh whole-job completion. The value is $\max_{\sum b_i \leq B} \sum_i G_i(b_i)$ independently of order. For the final statement, saturating every attempted observation uses at most $\Sigma + r \max_i \sigma_i$ writes on an admissible run and forces full work. Below that budget, reserve all r rejections for full observations of a maximum- σ_i job. Full work would require exactly that many writes, so the worst cost is strictly smaller. $\square$

For two and one unit singleton components, $r = 1$ gives $K(4) = 2$, yet an unaware policy optimal at $B = 1$ can be forced to pay 3 with four writes. Intermediate curves generally require the finite game.

4.2 Charging Every Call Changes the Boundary Again

Repair work omits the protected cost of validation itself. To expose this assumption, charge $\kappa \geq 0$ on *each* call and a common $\mu \geq 0$ once upon completion. Acceptance on call t costs $\mu + t\kappa + w(D)$.

Preparation remains outside this metric. Let M be the sorted multiset of qm numbers

$$\{t\kappa + G(k) : 1 \leq t \leq q, 0 \leq k < m\},$$

with one-based entries M_j and repeated entries retained.

THEOREM 8 (TOLL CURVE AND SIMULTANEOUS-OPTIMALITY BOUNDARY). *The known-budget optimum is $\mu + C_q(B)$, where*

$$C_q(B) = \begin{cases} \min\{\kappa + W^*, M_{B+1}\}, & B < qm, \\ \kappa + W^*, & B \geq qm. \end{cases}$$

For $q \geq 2$, simultaneous optimality holds exactly in the following cases: $\kappa = 0$ and $G(1) = W^$, or $\kappa \geq W^*$. It fails for every $0 < \kappa < W^*$. The common cost μ does not change these conditions. At $q = 1$, first acceptance is always simultaneously optimal.*

PROOF. Every completed play pays μ exactly once and rejects pay none, so remove that common constant. Fix $T < \kappa + W^*$. On call t , an unacceptable dirty set has $t\kappa + w(D) > T$. Its least write cost is $a_t(T) = \min\{k : t\kappa + G(k) > T\}$. Such an index exists since $G(m) = W^*$. A sequence of q unacceptable observations is possible exactly when $\sum_t a_t(T) \leq B$: each interval can independently realize a minimum-cost witness. Otherwise accepting the first acceptable observation finishes within q calls and cost T . Because G is nondecreasing, $a_t(T)$ counts indices $0 \leq k < m$ with $t\kappa + G(k) \leq T$. The omitted $k = m$ term exceeds T for every t . Thus $\sum_t a_t(T)$ counts all entries of M at most T , including ties. Feasibility requires this count to exceed B , first occurring at M_{B+1} if below the first-accept cap. For $B \geq qm$, the count cannot exceed B , leaving that cap. This proves both the index and endpoint.

If $\kappa > 0$ and a policy ever rejects a first observation, follow it by maximum damage until it accepts. At a sufficiently large finite budget it pays at least $2\kappa + W^*$, exceeding the aware first-accept cap $\kappa + W^*$. Thus a simultaneous optimum must accept every first observation. Let k^* be the first index with $G(k^*) = W^*$. With known budget k^* , accept all first observations except maximum damage; reject that one and the next observation is necessarily clean. Its worst cost is at most $\max\{\kappa + W_<, 2\kappa\}$, where $W_< < W^*$ is the largest attainable nonmaximum damage. For $0 < \kappa < W^*$ this is strictly smaller than the first-accept value $\kappa + W^*$. Conversely, when $\kappa \geq W^*$, presenting a first observation with damage $G(B)$ forces either $\kappa + G(B)$ on acceptance or at least 2κ on rejection. First acceptance attains the former, which is no larger. The zero-toll case is Theorem 6. $\square$

4.3 With Call Charges, Precedence and History Matter

Return to the workflow interface of Section 4.1, charging $\kappa > 0$ per call. Write $K^\kappa(B)$ for its known-budget value, $H(B) = \max_{\sum b_i \leq B} \sum_i G_i(b_i)$ and $C_{\max} = n\kappa + \sum_i W_i$.

THEOREM 9 (CALL CHARGES IN WORKFLOWS). *For independent jobs and $r \geq 1$, a simultaneous optimum exists iff $\kappa \geq \max_i W_i$. This condition is sufficient on every DAG: first acceptance has exact value $n\kappa + H(B)$. On a general DAG, a sink with $W_i > \kappa$ prevents simultaneous optimality. The condition on all jobs is no longer necessary.*

PROOF. For sufficiency, fix an allocation attaining $H(B)$. Present its dirty set on each job's first attempt and leave that job clean after a rejection. Accepting pays the allocated damage; rejecting incurs an extra toll at least as large. This matches the first-accept upper bound, regardless of order.

For necessity, a simultaneous optimum cannot reject after a prefix whose completed jobs were all fully dirty: saturating every remaining attempt would then force cost above $C_{\max}$ at some finite budget. Hence fully dirty first observations force it to pay $C_{\max}$ with Σ writes. If a job j with $W_j > \kappa$ can be placed last, an aware policy at budget Σ accepts preceding jobs. If any observation is less

than full it accepts throughout, costing strictly below $C_{\max}$. Otherwise all Σ necessary writes are spent; rejecting j once then completing cleanly costs $C_{\max} - W_j + \kappa < C_{\max}$. The finite outcome set makes the worst cost strictly smaller. Every independent job, and every DAG sink, can be last.

For the final claim, let A have singleton components of weights 1 and 2, let C have one component of weight 1, require A before C , and take $r = 1, \kappa = 2$. First acceptance has curve 4, 6, 7, 8 at $B = 0, 1, 2, \geq 3$. Matching lower bounds subtract the mandatory toll 4. At $B = 1$, dirty A 's weight-2 component: acceptance or rejection costs at least 2. At $B = 2$, do the same, then dirty C after acceptance (cost 3), or the weight-2 component again after rejection (cost 4). At $B = 3$, first saturate A with two writes; then dirty C after acceptance (cost 4), or the weight-2 component after rejection (cost 4). Thus first acceptance is a common optimum although $\kappa < W_A$. Reversing the order makes A a sink and removes every common optimum. $\square$

History-dependent rejection can be essential even with one-component jobs. Consider chain (a, h, a) , $h > a > 0$, with $r = 1, \kappa = a$. Accept throughout except that a dirty middle job is rejected once if the first job completed cleanly. Its exact curve is $3a, 4a, 4a + h, 5a + h$ at $B = 0, 1, 2, \geq 3$. First acceptance instead costs $3a + h > 4a$ at one write. For upper bounds, a retry leaves at most $B - 1$ writes; whenever its extra toll a is paid, the clean first job has saved a . For matching lower bounds, one dirty observation forces at least a beyond baseline. With two writes, leave the first job clean and dirty the middle, then dirty the last after acceptance or the middle again after rejection, forcing $a + h$. With three writes, dirty the first job: after acceptance apply that two-write strategy to the last two jobs; after rejection dirty the middle and last instead. This forces $h + 2a$. A preceding clean rejection already spends the toll and leaves no retry. No writer budget is supplied; $(a, h) = (1, 3)$ gives curve 3, 4, 7, 8.

Constructing Policies and Minimizing Uniform Loss. A finite recurrence decides whether a common optimum exists beyond these cases. Let $c(D)$ be the minimum footprint-word length realizing observation D on the attempted job. Set $E = \Sigma + r \max_i \sigma_i$; saturating every attempt proves $K^\kappa(b) = C_{\max}$ for $b \geq E$. Let e be the least-consistent write count clipped to E , t the remaining rejections, and a the cost already paid. Define $\Theta(\emptyset, t, e) = K^\kappa(e)$ and, with $e_D = \min(e + c(D), E)$,

$$\Theta(S, t, e) = \max_{i \in \mathcal{A}(S)} \min_D \max \{ \Theta(S - i, t, e_D) - \kappa - w_i(D), \Theta(S, t - 1, e_D) - \kappa \}, \quad (1)$$

omitting the rejection term at $t = 0$. A continuation exists exactly when $a \leq \Theta(S, t, e)$; in particular, $\Theta(J, r, 0) \geq 0$ decides common optimality.

To prove this, consider a completed history and sum the minimum footprint-word lengths of its observations. Fresh preparations and independent own-job writes realize those words in separate intervals, so the sum is the least budget consistent with that history. Every larger budget permits the same history by leaving writes unused. Since K^κ is nondecreasing, meeting its value at the least budget is necessary and sufficient for that history to meet every consistent target. Clipping preserves every extension because $\min(\min(e, E) + c(D), E) = \min(e + c(D), E)$. Below E it retains the exact least budget; at or above E every consistent comparator target is $C_{\max}$. Future observation sets depend on the job, not the hidden remaining budget, since one policy must cover all finite budgets. Thus clipping changes neither targets nor choices, even if $c(D) > \sigma_i$. Accumulated cost a is never clipped.

Induct on $|S| + t$. With no jobs left, feasibility is the interval $a \leq K^\kappa(e)$. For a fixed ready job and observation, accepting adds $\kappa + w_i(D)$ and rejecting adds κ ; either strictly decreases the induction measure. Each child's feasible costs therefore give a shifted downward interval. The choice after observing D takes the larger endpoint. Requiring success for every observation takes the minimum; choosing a job before that observation takes the maximum. This proves Equation (1)

with its required $\exists i \forall D \exists$ action order. A maximizing job and any satisfied child inequality supply an executable policy using history alone. If the initial interval excludes zero, the failed alternatives identify an adversarial finite observation path for each attempted controller choice.

For this repair interface, also at $\kappa = 0$, let $C(P, B)$ denote a universally capped unaware policy's worst cost. The minimum uniform additive loss is

$$\delta^* = \min_P \sup_{B \geq 0} \{C(P, B) - K^\kappa(B)\} = -\Theta(J, r, 0) \geq 0.$$

For a fixed P , each reachable terminal history is realizable at its least consistent budget; monotonicity of K^κ makes that budget its worst regret witness. Replacing every terminal target by $K^\kappa(e) + \delta$ shifts all thresholds by δ , since minimum, maximum and constant subtraction commute with the shift. Hence a loss allowance δ is feasible iff $0 \leq \Theta(J, r, 0) + \delta$. Shifted child inequalities extract an attaining policy. Finite job/observation alphabets and at most $n + r$ rounds make the policy minimum attained, before clipping. Fixed per-completion costs cancel from both policy and comparator.

For the preceding $A \rightarrow C$ example with $\kappa = 1$, the informed curve is 2, 3, 5, 5, 6 at $B = 0, 1, 2, 3, \geq 4$. An emitted policy has curve 2, 4, 5, 6, 6 and minimum uniform loss $\delta^* = 1$. At $\kappa = 2$ the loss is zero.

For each minimum footprint count, only its maximum attainable damage is needed in the adversarial alternatives. With I unfinished sets and $h = \max_i (m_i + 1)$, the table has $O(I(r+1)(E+1))$ entries and uses $O(In(r+1)(E+1)h)$ arithmetic operations, given profiles and the comparator. For a fixed order, $I = n + 1$ and there is one job choice. This removes accumulated-cost enumeration, not potentially exponential DAG states or footprint profiling. It is a specialization of finite minimax and information-state reasoning, not a new general dynamic-programming principle.

5 Actual External Writes in Java

Use Java 17's `ConcurrentHashMap` with one foreground thread, distinct nonremoved keys, non-null immutable identity-equal values and never-reused write identities. Each job updates its fixed key using own snapshots and captured parents. Kernels and key hash/equality/comparison methods are stable, side-effect-free and normally terminating. Replacement accepts identity or equals equality [15, 16]. The API specifies atomic compute with one callback invocation [14]; the inspected source executes existing-key callbacks under a bin monitor, including tree bins [16]. Different keys may share bins.

Implement Table 1 using `replace(key, expected, result)` and `compute`. Validation-only callbacks return the prepared result on match, otherwise the unchanged entry with a failure flag; cached callbacks recompute on mismatch, and fresh callbacks always compute under protection. Capture permanent milestones locally rather than rereading the map.

Callbacks never directly or recursively modify the map, as required by the API. Our model additionally excludes other-key reads, other-job kernels and guard work; multiple preparations remain allowed. Q counts API invocations, not monitor acquisitions; L counts only kernel work under protection. Internal transfer/treeification preserves stored value references and adds no modeled external write or kernel work. The monitor may be released before API return: abstract postpublication internal steps away and place intervening writes at the next gate. Callbacks retain no guard across calls. Scheduling and normal call completion are assumptions, not consequences of finite writing.

THEOREM 10 (RESOURCE BOUNDS FOR ACTUAL WRITES). *Under the stated API, monitor and scheduling assumptions, Theorems 1–2's exact call/protected-kernel-work bounds hold for Java programs using the listed operations, with separate or shared internal bins. This is a mathematical refinement argument for those operations, not a mechanically verified property of an entire JDK binary.*

PROOF. For the upper bounds, the serial policies freshly read and prepare after every cheap failure. Their read-to-comparison intervals, including cached completions, are disjoint; every mismatch contains a distinct successful write. Thus at most B mismatches occur. Every fresh/cached completing call completes exactly one job, and a cached mismatch performs precisely that job's kernel inside.

For the lower bounds, use the target-job adversaries of Section 3. Immediately before a chosen comparison acquires its monitor, a successful own-key write installs a fresh identity, invalidating every retained preparation for that job. The writer need not run inside the monitor: moving this write before acquisition preserves the selected operation and its mismatch outcome. A cheap call fails; a cached call recomputes inside; a fresh callback already incurs its whole kernel inside. Stale copies cannot complete cheaply, captured parent outputs persist, and no monitor survives the preceding call. Bin collisions therefore do not prevent the next chosen write. The prohibition on other-job work within a callback ensures it cannot complete several targets for one call. The same write counts and hypothetical extra-write argument apply. $\square$

Operational accounting. Native kernels fold captured parents and an immutable array of length $8w_i - |\text{Pred}(i)| > 0$, executing $8w_i$ selected operations. Separate counters record protected work and calls; allocation, waiting, initialization, writers and other API instructions are excluded. Scalar charges are supplied costs.

6 Source Integrations and Their Boundaries

Roslyn document updates without a writer budget. The `Workspace.cs` loop at commit 6c4a46a3 transforms a captured immutable solution outside a semaphore, validates identity under it and retries mismatches. Publication preserves notification ordering [6, 7]. We modify only the `SourceText` path. After r global failures, two-mode completion transforms the latest solution inside; three-mode completion prepares outside, then reuses on match or recomputes inside on mismatch. The selector receives no writer/price tables.

Replayability and safety under protection are separate obligations. The scratch document list must be cleared each attempt; source review discusses replay and event ordering [7]. Our initial two-mode patch deadlocked on same-text notification reentry; restoring the pre-lock identity no-op repairs it. Four later missing-document cases timed out when virtual `GetDocumentName` reentered an unrelated update while holding the semaphore. Patch04 detects an absent target and runs the original transformation on the same captured solution after releasing protection, preserving errors/exception handling. All 14 fixed error cases then complete. These defects belong to our patches.

RQ2 checks linked propagation, event payloads and snapshots. Lazy work, allocation and `VersionStamp` generation limit transfer to counts; custom loaders, changed-text reentry and arbitrary compiler behavior remain outside scope. No upstream numerical call cap or latency SLA was found. A prepared-rebase comparator uses existing `WithDocumentContentsFrom` when solution ID, related-document IDs and affected project states match; otherwise it uses three-mode fallback. An unrelated-project change can preserve prepared target contents, but merging them into the latest solution still requires protection. This fits neither whole-kernel recomputation nor zero-cost empty-set acceptance: guard, merge and invalidation work have no calibrated additive law.

7 Formal Corroboration and Operational Evaluation

RQ1 checks resource/information claims; RQ2 checks source preservation and costs. Inputs are fixed before execution; arrival reanalysis is post-outcome. All failures, timeouts, invalids and corrections remain archived.

An Apple M2/24 GiB runs Python 3.10.9/3.12.14 and OpenJDK 17.0.19; inspected map source matches the cited version but its binary was not rebuilt. Hashes bind inputs, code, commands and traces. Codex assisted design, checkers, source comparison, proofs, analysis and writing; Claude supplied separate proof/direction critiques and manuscript reviews. Measurements are archived executions. These are author-side checks, not mechanical proofs, independent third-party reproduction or human studies.

An *input* fixes graph, works and prices/footprints; a *root* also fixes budget and retry allowance. Native maxima are coordinatewise over finite replay groups, not necessarily attained together. Table 3 keeps overlapping studies separate.

7.1 RQ1: Bounded Corroboration of the Proofs

Table 3. Bounded checks with separate denominators and comparisons. Zero mismatches means agreement with the specified equations, bounds or expected outcomes; rejecting an intentional mutant is an expected outcome.

Obligation	Evidence and comparison
Whole-kernel frontier	Direct minimax versus formulas: zero mismatches on 265,629 roots from 5,421 DAG/weight inputs (75 DAG shapes).
Native implementations	Aware: 9,314 records, 1,024 replay groups. Unaware: 17,714 records, 1,280 groups. All expected outcomes and coordinate maxima agree.
Local/shared caps	Direct games versus canonical policies, protection formulas and call bounds: zero violations on 35,796 roots (4,464 conditions).
Whole-kernel call charges	Threshold cost formula and informed lower/upper and loss bounds: zero violations on 42,084 roots (5,688 conditions). Equality with the informed value is not required.
Single-job repair	Dirty-set game, damage DP and informed formula: zero mismatches on 160,080 cells (3,495 inputs).
Single-job call charges	Direct games versus curves: zero mismatches on 143,226 informed values; separate recurrence/certificates agree on 33,688 competitive values.
Shared-retry workflows	61,246 informed roots in 8,960 conditions; theorem checks pass. The one-policy search matches the criterion on all 6,208 tested conditions (two jobs $r \leq 2$, three jobs $r \leq 1$).
Charged workflows	216,282 informed game values in 37,248 conditions; theorem checks pass. Boolean and threshold existence methods agree on all 37,248 conditions.
Uniform repair loss	Full-history policy vectors versus negative threshold roots and emitted policies: zero mismatches on 540 exploratory and 5,360 fixed conditions.

The uniform-loss confirmation has 12,344 nondominated vectors, 40,640 budget coordinates and 2,256 positive-loss conditions. Its Cartesian composition uses every dirty mask without count clipping; four corrupted claims reject. The earlier exploration has 913 vectors, 2,820 coordinates and 202 positive-loss conditions. Neither measures native costs.

Native unaware selectors receive no budget, price or table state and test budgets through four; separate counters reconstruct W, L, Q . Archived suffix/work studies retain rare gains and counterexamples to general and intermediate-state optimality.

Shared-retry inputs cover ordered pairs of 24 one-/two-component types on the empty DAG and both chains, $r = 0, 1, 2, 3$; triples of four types cover all eight forward DAGs. Known-budget games select a ready job, observe damage, then accept/reject. The separate one-policy search uses accumulated cost and minimum consistent writes over all dirty histories, without taking saturation

as its answer. Both studies agree on every listed condition; each rejects four corrupted profiles. The positive-toll study includes seven cases requiring conditional rejection. Initial execution failures and corrections remain archived.

The action-before-comparison studies include 8,928 known-budget local-cap comparisons; each detects four changed-input/result controls. Their serial fresh-preparation games corroborate that class only. Lemma 1 and the proofs establish the broader retained-record/inspection lower bounds. A separate one-job comparison checks 11,232 budgets and 131,040 policy-budget values against direct informed games and complete serial policies; all agree with Proposition 5. Four changed-output/cap controls reject.

Threshold tables agree with all 37,248 existence answers and 120 further three-component cases. A checker importing no constructor verifies 123 tables from full dirty sets, rejects four altered certificates, and exhaustively checks two executable policies. Threshold construction uses 803,712 states versus 616,039 for boolean search: 30.5% more states despite removing accumulated-cost enumeration. Profile controls individually change $G(0)$, maximum damage, saturation count or an observation's minimum write count by one; they do not measure general defect coverage.

The supplement retains 21,028 auxiliary compilation comparisons, six scale timeouts (two compact-method), 168 changed-search units and 4,220 excess-cost outcomes; none calibrates Roslyn.

A controlled operational consequence. A Java probe uses four dependent kernels $(1, 2, 3, 5)s$, $r = 0, 1, 2$, and exactly $B = r$ initial failures. Both unaware policies have $Q = 4 + r$, $W = (11 + r)s$; two/three modes protect $11s/0$ iterations. A budget-aware two-mode policy could also avoid protection. During every completing callback, a background compute on another key signals readiness before entry; the foreground holds its bin until that signal. Shared/disjoint-bin layouts have the same hook. Four scales, three allowances, two layouts/modes and three JVMs give 4,320 measurements/1,440 warmups; all complete and match 12 output references. A cross-job checker rejects 12 corruptions. At the largest scale, summed waits fall from 26.8 ms to $31.2\mu s$ in shared bins. Disjoint differences are submicrosecond and sometimes reverse. All 24 cells remain. This forced collateral-blocking probe is not natural-arrival performance or a latency bound.

7.2 RQ2: What Does the Source Integration Preserve?

Source projection and semantic checks. Pinned net8 Roslyn assemblies run on a pinned .NET 10 RC runtime. Four projects yield 3,837 source files, 4,019 document occurrences and 40.8 MB; two package-external items were excluded before outcomes. Targets maximize sharing then size, and single-project size for the background. Updates append comments; internal source references remain.

The semantic grid covers caps, concentrated/distributed writers, linked targets, no-ops, equal-content distinct texts and absent/deleted targets. Four modes each have 58 cases plus four unmodified zero-write controls: all 236 agree, with 2,190 queued events, 1,253 publications and 2,653 snapshots. A separate checker binds immediate/queued payloads and callbacks to solutions, linked publication chains, 4,016 untouched text identities/versions and metadata. All 12 corruptions reject. Concentrated $B = 5, 6$ schedules were selected before outcomes and differ from earlier schedules.

Rebase passes 18 error-reentry and five same-text controls; patch04 passes 14 and 4. The original same-text deadlock and four missing-document timeouts remain. A 96-process study varies source, parse options, references and linked text across eight arms and cold/warm caches. All 2,556 project projections match declared source, symbol, diagnostic and emission expectations; eight corruptions reject. Workspace/direct compilations share Roslyn and cover authored programs.

Earlier timing and stronger source operations. Earlier forced/arrival studies retain all outcomes: three-mode foreground medians improve in four settings and worsen in seven. In 320 shared- $r = 2$

batches, two modes use 10,802 calls and 7,131 protected transformations; three use 10,786 and 3,641, but raise total transformations to 14,427. Background p95 improves at 100 μ s, while burst foreground time worsens.

Rebase retains 4,219 protected merges at $r = 0$ and 4,528 at $r = 2$. A batch-content version passes the same conformance cases. Its 1,728 arrival units give mixed batch/sequential ratios: 0.906–1.189 foreground, 0.511–1.404 background p95, 0.917–1.281 observed protected spans. A document-state guard changes four of 96 compiler-case counter rows to reuse after reference removal, preserving newer references and diagnostics. All 288 common snapshot projections agree; one fewer warm snapshot gives 2,552 project checks. Timing of this document-state-guard variant is unmeasured; protected merge work remains.

Comparing shared and per-update contracts. The local baseline resets retries before each update in unchanged patch04. Eight arms cover unmodified baseline, instrumented original, shared two/three at $r = 2$ and local two/three at $r = 1, 2$. Eight cyclic Latin blocks balance process positions. Each process serves 32 serial updates and 64 FIFO writers with precomputed texts, no supplied budget, and intervals $d \in \{0, 10, 100, 1000\}$ μ s. Two warm-up and ten measured batches per interval are shuffled before execution. All 2,560 measured and 512 warm-up units pass source checks; 12 corruptions reject. Arrivals are authored.

For an update’s actual write count B_i , local modes satisfy $Q_i = 1 + f_i$, $f_i \leq \min(B_i, r)$, and inside count at most $1[B_i \geq r]$ (two) or $1[B_i \geq r + 1]$ (three). Per-update deltas sum to batch counts. All updates and batches meet Proposition 3’s count bounds. Table 4 reports every interval with separate medians, which need not coexist on one execution; B_{fg} counts foreground-period writes.

Table 4. New Roslyn cap comparison, 80 measured batches per cell. Entries are median $Q/B_{\text{fg}}/N_{\text{inside}}$: counted calls, foreground-period writes and full protected transformations. The universal call cap differs across rows.

Policy	Q cap	$d = 0$	10	100	1000 μ s
Shared two, $r = 2$	34	34/3/32	34/3/32	34/2/31	33/1/0
Shared three, $r = 2$	34	34/34/30	34/27/22	34/8/6	33/1/0
Local two, $r = 1$	64	62/31.5/30	54/24/22	40/9/8	33/1/1
Local three, $r = 1$	64	58/64/25	53/50.5/19	37/7/1	33/1/0
Local two, $r = 2$	96	79.5/64/23.5	77/57/22	39/7.5/1	33/1/0
Local three, $r = 2$	96	66/64/17	64.5/64/16	38/7/0	33/1/0

At 100 μ s, local three-mode $r = 2$ has median 38 calls and zero protected transformations, versus shared $r = 2$ ’s 34 and 6. Their caps differ (96/34), so this shows a resource tradeoff. B_{fg} depends on completion time; post-outcome reconstruction checks it and count bounds on 7,392 earlier batches, without a common controlled write budget.

Shared three/two foreground ratios at 0/10 μ s are 1.72 [1.40,2.11] and 1.58 [1.17,2.16]; both slower-arrival intervals and all writer-response intervals include 1. Counter snapshots lie inside foreground duration. Original/baseline ratios are 0.83–1.08 with all intervals including 1, leaving instrumentation overhead unidentified. Intervals are descriptive and multiplicity-unadjusted; eight blocks limit between-process resolution, and resampling cannot remove host/order effects. Native execution took 237.84 seconds; setup/build and analysis costs are archived.

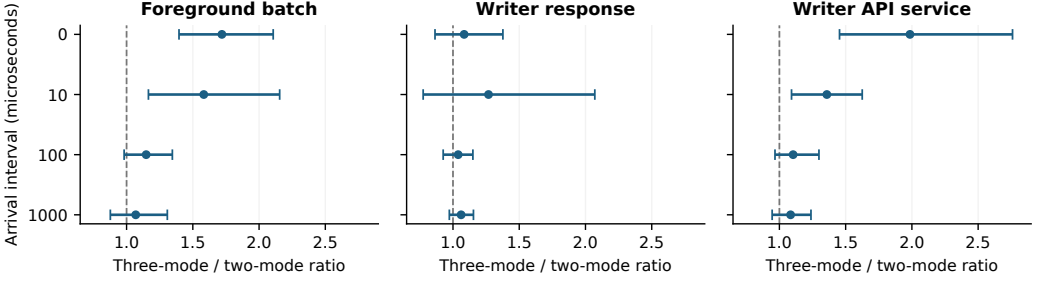


Fig. 3. New shared-cap timing comparison, $r = 2$. Points are geometric means of eight paired block ratios of process medians; bars are 95% percentile intervals from 10,000 fixed block-bootstrap resamples. Ratios above 1 favor two modes. Writer response is intended arrival to return; service is invocation to return. Each batch contributes its median across 64 writer requests. All four intervals and three metrics are shown.

8 Related Work and Research Lineage

Fallback and online contention management. Kung and Robinson [5], Section 3.3/p.220, already retain the critical-section semaphore during reexecution after repeated validation failure. Lock-elision retry caps likewise precede our work [22, 23]. Cached completion here requires an unrolled-back preparation and atomic identity-check/recompute; HTM capacity or explicit aborts need not consume a write [24]. RPC retry budgets may instead cap concurrency or time-window rates and permit failure [31, 32]. Those controls do not impose our persistent-job completion and total-call contract.

Welc, Saha and Adl-Tabatabai [33], Sections 3.1/7.1, already preserve and protect a running transaction's prefix during irrevocable transition. Their second-abort heuristic continues revocably if token acquisition fails, giving no universal attempt cap. Prefix reuse and irrevocability are inherited; our entryptpoint-specific resource curves are different guarantees.

Attiya et al. [20], Sections 2–3/5, analyze one Greedy contention manager that does not receive the per-job non-conflict failure bound k . It is $O(ks)$ -competitive for makespan with s resources, with a corresponding deterministic lower bound under time-dependent resource requests. Unknown-disturbance control and all-policy lower bounds thus predate ours. Our bounds instead concern global external writes, universal call caps and exact protected work for persistent jobs. Alistarh et al. [21], Sections 4–5, optimize whether and when to abort a conflicting transaction with known abort cost and unknown remaining duration. Their throughput policy lacks progress; Section 7 adds probabilistic progress, distinct from our universal call cap.

Guerraoui, Herlihy and Pochon [3], Sections 3–4, establish a priority-based abort/wait manager with bounded-delay progress and a makespan guarantee against an offline scheduler. Diegues and Romano [4], Sections 4–6 and its errata, adapt attempt budgets and capacity-abort handling using per-atomic-block measurements in GCC. Their progress/cycle objectives differ from our persistent preparations and hard local/shared call caps; their measurements do not calibrate our toll.

Synchronization synthesis and robust control. Černý et al. [11] choose retries, guards and caches in partial concurrent programs with a scheduler and weighted limit-average safety objective. Their Theorem 6 uses quantitative probabilistic bisimulation; the erratum corrects complexity to PSPACE. Later work optimizes protected-statement counts and statistically profiles acquisition/contention [12]; preemption-safety synthesis also addresses deadlock-free lock placement [13].

Finite n, B, r instances admit general-game encodings; our archived initial-total-cost potential proves neither source equivalence nor intermediate-state quotients.

Mannor et al. [8] supply global-deviation-budget DP; Postek et al. [17] give finite-candidate DP and causal budgeted routing. Information-state minimax control [25] also precedes our recurrence. Our specialization proves that independently realizable intervals and monotonicity reduce repair feasibility to a minimum-consistent-write path; backward induction itself is inherited.

Online algorithms with predictions. Karlin et al. [34], Section 4/Theorem 4.1, use cost-balancing counters for competitive bus communication against an offline comparator. Our call-capped additive loss and causal budget-informed comparator differ. Purohit, Svitkina and Kumar [27], Algorithm 2/Theorem 2.2, trade consistency against robustness using predicted ski days and a trust parameter. Lykouris and Vassilvitskii [28], Sections 2–3, bound expected cache misses against an offline optimum across next-request prediction errors. Wei and Zhang [29], Theorems 1.1–1.2, give tight skirental tradeoffs. Our comparator knows only the write bound and remains causal. We characterize one-write saturation for a no-advice policy equaling this informed minimax curve at every budget under a universal cap.

Incremental computation and repair costs. Mokhov, Mitchell and Peyton Jones [1], Definitions 2.1/3.1 and Sections 4–6, classify build schedulers/rebuilders using persistent dependencies; Section 6.6 bounds iterative execution. Ley-Wild, Acar and Fluet [2], Theorems 4–5, relate consistent change propagation to trace-distance cost. Adaption [30], Theorem 4.3/Algorithm 1, ensures nonincremental-result consistency and demand-driven propagation; dirty-edge value checks can avoid recomputation. These guarantees differ from our fixed footprint-union cost and the choice to accept damage or spend a call on fresh preparation. They do not state our acceptance value or common-policy boundary.

9 Limitations and Threats to Validity

The whole-kernel contract excludes replenishment, structural change, exceptions, external completion and retained guards. Retained charged guards improve five of 384 inputs (e.g., 242 versus 243), and one intermediate state improves from 45 to 43. Equality for the stronger interface remains unproved.

Selected Roslyn paths and shared-compiler checks do not establish arbitrary concurrent equivalence. Our patch deadlocks/timeouts distinguish replayability from safe protected execution. Rebase costs remain uncalibrated; general footprint profiling may be exponential and native validation may reveal different information.

One-workstation authored arrivals, instrumentation and cache/JIT effects limit timing inference; human benefit is unmeasured. Archived scalar gains occur in 2/96 cells, timing fits succeed in 2/8, and all-budget compilation can exceed requested-value costs. Finite tests and AI-assisted inspection are neither mechanical proofs nor independent reproductions.

10 Conclusion

Cached completion attains the specified interface’s whole-kernel frontier under one universal call cap, at possible total-work and foreground-time costs. Partial repair and call charges change the information boundary; finite synthesis also minimizes uniform additive loss when no common optimum exists. Source checks expose preservation obligations and count tradeoffs; timing benefits depend on costs and contention.

Data Availability

The public supplement [19] supplies inputs, code, all outcomes/corrections, derivations and replay commands. Central proofs are in this paper; priced results are auxiliary and zero-fee evidence [18] is unchanged. Hashes record path projections. Modified sources retain notices; upstream archives, runtimes, articles and private correspondence are excluded.

References

- [1] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. 2018. Build Systems à la Carte. *PACMPL* 2, ICFP, Article 79, 1–29. <https://doi.org/10.1145/3236774>. Author version: <https://simon.peytonjones.org/assets/pdfs/build-systems-original.pdf>.
- [2] Ruy Ley-Wild, Umut A. Acar, and Matthew Fluet. 2009. A Cost Semantics for Self-Adjusting Computation. In *POPL*, 186–199. <https://doi.org/10.1145/1480881.1480907>. Author version: <https://ttic.uchicago.edu/~pl/sa-sml/popl09.pdf>.
- [3] Rachid Guerraoui, Maurice Herlihy, and Bastian Pochon. 2005. Toward a Theory of Transactional Contention Managers. In *PODC*. Author version. <https://lpdwww.epfl.ch/upload/documents/publications/neg--509904679f181-pochon.pdf>.
- [4] Nuno Diegues and Paolo Romano. 2014. Self-Tuning Intel Transactional Synchronization Extensions. In *ICAC*, 209–219. <https://www.usenix.org/conference/icac14/technical-sessions/presentation/diegues>. Errata.
- [5] H. T. Kung and John T. Robinson. 1981. On optimistic methods for concurrency control. *ACM Transactions on Database Systems* 6, 2, 213–226. <https://doi.org/10.1145/319566.319567>.
- [6] .NET Foundation. 2025. *Roslyn source*, commit 6c4a46a31302167b425d5e0a31ea83c9a9aa1d09. Workspace.cs, Solution.cs, SolutionCompilationState.cs and SolutionState.cs. Accessed September 10, 2026. <https://github.com/dotnet/roslyn/tree/6c4a46a31302167b425d5e0a31ea83c9a9aa1d09>.
- [7] .NET Foundation contributors. 2020. Add SetCurrentSolution with solution transformation, pull request 42228. Source review on replayability and event ordering. Accessed September 10, 2026. <https://github.com/dotnet/roslyn/pull/42228>.
- [8] Shie Mannor, Ofir Mebel, and Huan Xu. 2012. Lightning Does Not Strike Twice: Robust MDPs with Coupled Uncertainty. In *ICML*. <https://icml.cc/2012/papers/215.pdf>.
- [9] Awi Federgruen and Henri Groenevelt. 1986. The Greedy Procedure for Resource Allocation Problems: Necessary and Sufficient Conditions for Optimality. *Operations Research*. https://business.columbia.edu/sites/default/files-efs/pubfiles/4071/federgruen_greedy_procedure.pdf.
- [10] Dimitris Bertsimas and Melvyn Sim. 2004. The Price of Robustness. *Operations Research* 52, 1, 35–53. <https://www.mit.edu/~dbertsim/papers/Robust%20Optimization/The%20price%20of%20Robustness.pdf>.
- [11] Pavol Černý, Krishnendu Chatterjee, Thomas A. Henzinger, Arjun Radhakrishna, and Rohit Singh. 2011. Quantitative Synthesis for Concurrent Programs. In *CAV*, 243–259. https://doi.org/10.1007/978-3-642-22110-1_20. Author publication page and complexity erratum: <https://www.microsoft.com/en-us/research/publication/quantitative-synthesis-for-concurrent-programs/>.
- [12] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2015. Optimizing Solution Quality in Synchronization Synthesis. *arXiv preprint arXiv:1511.07163v1*. <https://arxiv.org/abs/1511.07163>.
- [13] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2017. From non-preemptive to preemptive scheduling using synchronization synthesis. *Formal Methods in System Design* 50, 97–139. <https://doi.org/10.1007/s10703-016-0256-5>.
- [14] Oracle. 2021. ConcurrentHashMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html>.
- [15] Oracle. 2021. ConcurrentMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentMap.html>.
- [16] OpenJDK. 2026. ConcurrentHashMap.java, jdk17u source, commit 982d53c227943a511e8d61694feced764b71a6c1. Accessed September 7, 2026. <https://github.com/openjdk/jdk17u/blob/982d53c227943a511e8d61694feced764b71a6c1/src/java.base/share/classes/java/util/concurrent/ConcurrentHashMap.java>.
- [17] Krzysztof Postek, Ward Romeijnders, and Wolfram Wiesemann. 2025. Technical Note: Multi-Stage Robust Mixed-Integer Programming. *Operations Research* 73, 6, 3345–3358. <https://doi.org/10.1287/opre.2023.0520>. Online appendix; author code.
- [18] Anonymous Authors. 2026. *Compiling Optimal Retry Policies for Dependent Transformations: Research Artifact*. Version 5 with publication supplement, commit e0e0110. <https://github.com/research-artifact-archive/research-artifact/tree/e0e011023c0591bbbf422390ad8d29785ad0023f>.
- [19] Anonymous Authors. 2026. *Retry Synthesis with Bounded Calls and Protected Work: Research Artifact*. Version 19 one-job minimax regret and repair-work comparison, commit c49ae8b. <https://github.com/research-artifact-archive/research-artifact/tree/b606568483b914dcf04216c381ee0e48d44d8207>.

- [20] Hagit Attiya, Leah Epstein, Hadas Shachnai, and Tami Tamir. 2006. Transactional Contention Management as a Non-Clairvoyant Scheduling Problem. In *PODC*. <https://cs.idc.ac.il/~tami/Papers/podc06.pdf>.
- [21] Dan Alistarh, Syed Kamran Haider, Raphael Kübler, and Giorgi Nadiradze. 2018. The Transactional Conflict Problem. *arXiv preprint* arXiv:1804.00947v1. <https://arxiv.org/abs/1804.00947>.
- [22] Free Software Foundation. 2025. GNU C Library 2.42: x86 lock elision, revision d2097651. `elision-lock.c` and `elision-conf.c`. <https://github.com/bminor/glibc/tree/d2097651cc57834dbfcaa102ddfaca0d86cfb66/sysdeps/unix/sysv/linux/x86>. Official tunables: https://www.gnu.org/software/libc/manual/html_node/Elision-Tunables.html.
- [23] OpenJDK. 2025. HotSpot 21.0.8+9 RTM monitor code, revision 54f20959. `c2_MacroAssembler_x86.cpp` and `globals_x86.hpp`. <https://github.com/openjdk/jdk21u/tree/54f2095960f01d957f2335cafa0defb956e13a2c/src/hotspot/cpu/x86>.
- [24] GNU Compiler Collection. n.d. x86 Transactional Memory Intrinsics. Accessed September 10, 2026. <https://gcc.gnu.org/onlinedocs/gcc/x86-transactional-memory-intrinsics.html>.
- [25] Aditya Dave, Nishanth Venkatesh, and Andreas A. Malikopoulos. 2024. Approximate Information States for Worst-Case Control and Learning in Uncertain Systems. *arXiv preprint* arXiv:2301.05089v2. <https://arxiv.org/abs/2301.05089>.
- [26] Zvi Lotker, Boaz Patt-Shamir, and Dror Rawitz. 2012. Rent, Lease, or Buy: Randomized Algorithms for Multislope Ski Rental. *SIAM Journal on Discrete Mathematics* 26, 2, 718–736. <https://doi.org/10.1137/100794018>.
- [27] Manish Purohit, Zoya Svitkina, and Ravi Kumar. 2018. Improving Online Algorithms via ML Predictions. In *NeurIPS* 31. <https://proceedings.neurips.cc/paper/2018/hash/73a427badebe0e32caa2e1fc7530b7f3-Abstract.html>.
- [28] Thodoris Lykouris and Sergei Vassilvitskii. 2018. Competitive Caching with Machine Learned Advice. In *ICML*, PMLR 80, 3296–3305. <https://proceedings.mlr.press/v80/lykouris18a.html>.
- [29] Alexander Wei and Fred Zhang. 2020. Optimal Robustness-Consistency Trade-offs for Learning-Augmented Online Algorithms. In *NeurIPS* 33. <https://proceedings.neurips.cc/paper/2020/file/5bd844f11fa520d54fa5edec06ea2507-Paper.pdf>.
- [30] Matthew A. Hammer, Khoo Yit Phang, Michael Hicks, and Jeffrey S. Foster. 2014. Adapton: Composable, Demand-Driven Incremental Computation. In *PLDI*. <https://doi.org/10.1145/2594291.2594324>. Author version: <https://www.cs.tufts.edu/~jfoster/papers/pldi14.pdf>.
- [31] Mike Ulrich. 2016. Addressing Cascading Failures. In *Site Reliability Engineering*, Chapter 22, Retries. <https://sre.google/sre-book/addressing-cascading-failures/>.
- [32] Envoy contributors. n.d. `CircuitBreakers.Thresholds.RetryBudget`. Accessed September 10, 2026. https://www.envoyproxy.io/docs/envoy/latest/api-v3/config/cluster/v3/circuit_breaker.proto.html.
- [33] Adam Welc, Bratin Saha, and Ali-Reza Adl-Tabatabai. 2008. Irrevocable Transactions and their Applications. In *SPAA*, 285–296. <https://doi.org/10.1145/1378533.1378584>.
- [34] Anna R. Karlin, Mark S. Manasse, Larry Rudolph, and Daniel D. Sleator. 1988. Competitive Snoopy Caching. *Algorithmica* 3, 79–119. <https://doi.org/10.1007/BF01762111>.